

Функциональные требования
Архитектура ПО / Software Architecture
Functional Requirements (*FR*)

К концу вы сможете:

1. Отличить **функциональные требования** (*functional requirements, FR*) от решений и UI-деталей
2. Писать FR в формах **shall-statement** и **user story** (*user story*)
3. Добавлять **критерии приёмки** (*acceptance criteria*), по которым требование проверяемо
4. Находить типичные ошибки: размытые глаголы, «неизмеримость», смесь с NFR

1. Что такое FR (и чем не является)
2. Формы записи: shall / user story
3. Хорошие vs плохие FR — много примеров и переписываний
4. Критерии приёмки
5. Мини-кейсы: банк, магазин, чат, портал, IoT
6. Частые ошибки и связь с архитектурой

Функциональное требование (*functional requirement*)

Описывает **поведение** системы: что она должна *уметь сделать* для пользователя или другой системы (данные принять, правило применить, результат выдать).

- Фокус: **что** (*what*), не *на каком фреймворке*
- Проверяемо: можно придумать тест / сценарий приёмки
- Обычно видно в use case, user story, спецификации «The system shall. . . »

FR vs решение vs качество

Тип	Пример	Комментарий
FR	Перевести деньги между счетами	Поведение
Решение (<i>design</i>)	Использовать Kafka	Как реализовать — не FR
NFR / качество	р95 перевода < 2 с	<i>Как хорошо</i> — следующая лекция

Запомните

FR отвечает: «Система делает X». NFR отвечает: «При каких условиях / с каким качеством».

- Бэклог / user stories
- Спецификация требований (*SRS*) (раздел Functional)
- Use cases / сценарии использования
- Контракты API («endpoint делает...») — часто *следствие* FR

Архитектору FR нужны, чтобы понять **границы системы** и потоки данных — не чтобы писать код кнопок.

Викторина 1 — FR или нет?

Викторина

Какое утверждение — функциональное требование?

1. Система должна быть современной и удобной
2. Система должна позволять студенту загрузить файл домашки до дедлайна
3. Использовать React и PostgreSQL
4. Время ответа API p95 < 200 мс

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

2. Поведение: загрузка до дедлайна.

(1) размыто; (3) решение; (4) NFR (performance).

Классика спецификаций:

Шаблон

The system shall <действие> <объект> <условия/ограничения поведения>.

Плохо: The system shall support payments.

Лучше: The system shall allow a registered customer to pay an open cart with a saved card and receive an order confirmation.

Шаблон

Как <роль>, я хочу <действие>, чтобы <ценность>.

- Хорошо для обсуждения с заказчиком
- Сама по себе часто **неполная** — нужны acceptance criteria
- Не заменяет архитектурный анализ границ

Одна и та же FR — две формы

Story: Как преподаватель, я хочу выставить оценку за работу, чтобы студент увидел результат в журнале.

Shall: The system shall allow an authenticated instructor to assign a numeric grade (0–100) to a submitted assignment and make that grade visible to the owning student.

Оба — FR; shall обычно точнее для проверки.

1. **Атомарность:** одна способность, не «и ещё десять»
2. **Ясный актер / триггер:** кто инициирует
3. **Наблюдаемый результат:** что изменилось в системе / для пользователя
4. **Проверяемость:** можно принять или отклонить на тесте
5. **Без лишнего UI-диктата:** не «синяя кнопка справа»
6. **Без скрытого NFR:** не подменять «быстро» без меры

Переписывание 1 — банк

Плохо: Система должна уметь работать с деньгами.

Плохо: Красивый экран перевода.

Лучше (FR): Система должна позволять клиенту с двухфакторной аутентификацией создать перевод на счёт внутри банка, указав сумму и назначение, и получить статус «принят / отклонён / на проверке».

Переписывание 2 — e-commerce

Плохо: Корзина как у Amazon.

Плохо: Система использует Redis для корзины.

Лучше: Зарегистрированный покупатель может добавить товар в корзину, изменить количество, удалить позицию и оформить заказ из текущего состава корзины.

Redis — решение; «как у Amazon» — непроверяемый лозунг.

Переписывание 3 — чат

Плохо	Лучше (FR)
Чаты в реальном времени	Пользователь А отправляет текстовое сообщение в диалог с В; В видит сообщение в истории диалога (при онлайн — без ручного обновления страницы)
Поддержка файлов	Участник диалога может прикрепить файл ≤ 25 МБ типов PDF/JPG/PNG

Переписывание 4 — кампусный портал

Плохо: Удобная сдача работ.

Лучше: Студент может загрузить один или несколько файлов к заданию до наступления дедлайна; после дедлайна загрузка отклоняется с понятной причиной.

Отдельная FR: Преподаватель может скачать архив всех сдач группы по заданию.

Переписывание 5 — IoT / умный дом

Плохо: Умный термостат должен быть интеллектуальным.

Лучше: Пользователь задаёт целевую температуру и расписание; контроллер поддерживает режим «дом / ушёл» и передаёт команды на исполнитель отопления.

Отдельная FR: При потере связи с облаком локальный контроллер продолжает выполнять последнее сохранённое расписание.

Последний пункт — всё ещё поведение (функция деградации), не «магия AI».

Таблица: глаголы-убийцы

Избегать	Почему	Замена
support / обеспечивать	слишком широко	конкретное действие
handle / обрабатывать	что на выходе?	принять / отклонить / сохранить
manage / управлять	пачка FR в одном	разрезать на атомы
allow users to easily. . .	UI + NFR в одном	FR + отдельно usability NFR

Викторина

«Система должна поддерживать уведомления» — главная проблема?

1. Это слишком атомарно
2. Глагол «поддерживать» размыт: какие уведомления, кому, по какому событию?
3. Это обязательно NFR про performance
4. Нельзя писать FR про уведомления

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

2. Нужно уточнить событие, канал, получателя, содержимое.

Пример: «При смене статуса заказа на „готов“ система отправляет push владельцу заказа».

Зачем

FR без критериев приёмки (*acceptance criteria*) легко «сдать» с разным пониманием готовности.

Форматы:

- Список Given / When / Then (*Gherkin-style*)
- Нумерованный чек-лист наблюдаемых фактов
- Примеры вход→выход (для правил)

Пример — оплата заказа

FR: Покупатель оплачивает заказ сохранённой картой.

Acceptance:

Given открытый заказ и сохранённая карта, When подтверждает оплату, Then статус заказа = `paid`, корзина очищена.

Given платёжный провайдер вернул `decline`, Then заказ = `payment_failed`, позиция на складе не резервируется повторно.

Повторный клик «Оплатить» не создаёт второй успешный платёж за тот же заказ.

Пример — роли на портале

FR: Только преподаватель курса выставляет оценки по своим заданиям.

Acceptance:

Студент не видит UI и API для выставления оценки (403/скрытие).

Преподаватель курса А не может оценить задание курса В.

В журнале аудита есть запись: кто, какую оценку, когда.

Аудит — часто отдельная FR (или NFR compliance); здесь усиливает проверяемость.

Викторина

Какой критерий приёмки сильнее для FR «сброс пароля по почте»?

1. «Должно быть удобно»
2. «Использовать SendGrid»
3. «Ссылка в письме одноразовая, истекает через 30 мин; после сброса старые сессии пользователя завершаются»
4. «Пароль должен быть безопасным» без правил

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

3. Наблюдаемые правила поведения.

(2) — решение; (1)(4) — размытые качества без меры.

Кейс А — Необанк: список FR (фрагмент)

Открыть счёт после KYC-статуса approved

Показать баланс и ленту транзакций владельцу

Создать перевод SEPA с лимитом на сутки

Заблокировать карту из приложения

Получить PDF-выписку за выбранный месяц

Попробуйте

Для пункта 3 добавьте 2 acceptance criteria (успех / отказ по лимиту).

Кейс В — Маркетплейс: разрезать «управлять заказами»

Размыто: «Система управляет заказами».

Атомы FR:

- Создать заказ из корзины
- Отменить заказ до передачи в доставку
- Продавец отмечает «отгружен» + трек-номер
- Покупатель открывает спор / возврат

Каждое — отдельная история и тесты.

- **FR:** Доставка сообщения в историю диалога; статусы sent / delivered / read
- **NFR (позже):** задержка доставки $p95 < 1$ с при онлайн-получателе

Частая ошибка

Писать в FR только «real-time», без поведения статусов и хранения истории.

Кейс D — IoT-теплица

FR

Считать влажность каждые 5 мин и писать в журнал

Включить полив, если влажность < порога

Оператор задаёт порог в приложении

He FR

«База должна быть быстрой»

«Использовать MQTT брокер X»

«Красивый дашборд»

1. Размытые глаголы (support, manage, handle)
2. UI-диктат вместо поведения («красная кнопка»)
3. Смесь нескольких фич в одном требовании
4. Подмена FR решением (стек, паттерн, вендор)
5. Непроверяемые формулировки без acceptance
6. Смешение с NFR («безопасно», «быстро») без сценария

Зачем архитектору чистые FR

- Понять **use cases** и границы системы
- Выявить интеграции (платежи, почта, IdP)
- Оценить данные и модули (неделя про модульность)
- Отделить обязательное поведение от «хотелок качества»

Плохие FR → плохие границы модулей и вечные споры на приёмке.

Викторина

«Покупатель получает email с номером заказа после успешной оплаты» — это:

1. Функциональное требование (поведение уведомления)
2. Только performance NFR
3. Только выбор SMTP-провайдера
4. Не требование, а баг

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

1. Это FR про уведомление по событию.

Скорость доставки письма / SLA почты можно добавить отдельным NFR.

Викторина

Верно или неверно?

«Система должна использовать микросервисы» — функциональное требование.

1. Верно
2. Неверно — это архитектурное / design-решение

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ

Неверно. Стил ь развёртывания — не описание пользовательского поведения.

1. FR = наблюдаемое поведение системы
2. Пишите атомарно: shall и/или user story + acceptance
3. Переписывайте размытое на конкретные действия и результаты
4. Не смешивайте UI-диктат, стек и NFR внутрь FR
5. Чистые FR кормят архитектуру и приёмку

функциональное требование (*functional requirement, FR*)
shall-statement · user story · критерии приёмки (*acceptance criteria*)
актор · проверяемость · атомарность

Следующая лекция

Нефункциональные требования (*NFR*) / атрибуты качества: измеримые сценарии, конфликты, контраст FR vs NFR

Практика

Раздатка Markdown: переписать плохие FR, написать acceptance, отличить FR от NFR

Что ещё мутно: формулировки FR, acceptance или граница с NFR?